

Universidade Federal
de Campina Grande



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
DEPARTAMENTO DE ENGENHARIA ELÉTRICA E INFORMÁTICA
PROGRAMA DE RESIDÊNCIA EM MICROELETRÔNICA
MCTI/Softex/IRede

RELATÓRIO TÉCNICO

Semana 4 - Expansão de chave e datapath AES-128

Aluno: Guilherme José Mendes Jerônimo
Professor: Prof. Henrique Cunha
Semestre: 2026.2

Campina Grande - PB
2026

Sumário

1 Objetivo **1**

2 Arquivos entregues **1**

3 Expansão de chave (`minisoc_aes_key_exp`) **1**

4 Datapath (`minisoc_aes_dp`) **2**

4.1 Interface de controle 2

4.2 Escalonamento (1 rodada por ciclo) 2

5 Verificação **3**

5.1 Estratégia 3

5.2 Vetores de referência (todos conferem) 3

5.3 Robustez do *testbench* 3

5.4 *Lint* 4

5.5 Como reproduzir 4

6 Decisões de projeto e consequências para as próximas semanas **4**

7 Limitações **4**

8 Conclusão **5**

1 Objetivo

Implementar e verificar `minisoc_aes_key_exp` (expansão da chave AES-128, 11 *round keys*) e `minisoc_aes_dp` (cifra e cifra inversa de um bloco de 128 bits), de acordo com a microarquitetura iterativa definida na Semana 3, e conferir ambos com os vetores da FIPS-197 nos dois sentidos.

2 Arquivos entregues

A Tabela 1 lista os módulos RTL, *testbenches* e *scripts* desenvolvidos e integrados nesta etapa.

Tabela 1: Arquivos entregues na Semana 4.

Arquivo	Descrição
`rtl/minisoc_aes/minisoc_aes_key_exp.sv`	Expansão de chave pré-computada, sequencial
`rtl/minisoc_aes/minisoc_aes_dp.sv`	Datapath iterativo, 1 rodada por ciclo, cifra e cifra inversa
`rtl/minisoc_aes/minisoc_aes_gf_pkg.sv`	Funções de GF(2^8) (`xtime` e multiplicações por constantes)
`tb/tb_aes_key_exp.sv`	*Testbench* das 11 *round keys*
`tb/tb_aes_dp.sv`	*Testbench* do datapath (FIPS-197 B e C.1 + 64 vetores aleatórios)
`scripts/gen_aes_ref.py`	Modelo de referência Python (validado contra a biblioteca `cryptography`) e gerador de vetores
`scripts/run_tests.sh`	Comando único: *lint* + simulação de todos os *testbenches*

As transformações de rodada (`sbox`, `sub_bytes`, `shift_rows`, `mix_columns`, `add_round_key` e inversas) são as das Semanas 2 e 3 e são instanciadas sem alteração pelo `dp`.

3 Expansão de chave (`minisoc_aes_key_exp`)

Escolha: pré-computada. As 11 *round keys* são calculadas uma vez por `START` e guardadas em um banco 11×128 bits, lido combinacionalmente por `i_rk_sel`.

- **Decriptação:** a cifra inversa consome as *round keys* na ordem $10 \rightarrow 0$. Com o banco pré-computado, o acesso é direto. Em uma implementação *on-the-fly*, seria necessário percorrer a expansão inteira até a última chave e depois implementar a expansão inversa (ou refazê-la a cada bloco).

- **Área:** o custo é ≈ 1408 *flip-flops* (11×128) contra 128 da alternativa *on-the-fly*. Em contrapartida, o cálculo usa apenas **4 S-boxes** compartilhadas e uma rodada de expansão por ciclo.
- **Desempenho:** 10 ciclos de latência, amortizados em até 256 blocos (a chave é capturada uma vez no `START`, seção 8.3 do *Block Guide*). A S-box sai do caminho crítico da rodada do datapath.

Operação. Em `i_start` (com `o_busy = 0`) a chave é gravada em `rk[0]`. A cada ciclo seguinte calcula-se `rk[i]` a partir de `rk[i-1]`:

$$t = \mathrm{SubWord}(\mathrm{RotWord}(w_3)) \oplus \mathrm{Rcon}[i], \quad w'_0 = w_0 \oplus t, \quad w'_1 = w_1 \oplus w'_0, \quad w'_2 = w_2 \oplus w'_1, \quad w'_3 = w_3 \oplus w'_2.$$

`Rcon` é gerado por `xtime` (`01`, `02`, `04`, ..., `36`). O sinal `o_ready` sobe após a 10ª rodada.

4 Datapath (`minisoc_aes_dp`)

4.1 Interface de controle

A Tabela 2 descreve os sinais de entrada e saída do módulo `minisoc_aes_dp`.

Tabela 2: Interface de controle do módulo `minisoc_aes_dp`.

Sinal	Dir.	Descrição
`clk`, `rst_n`	I	*Clock* e *reset* assíncrono ativo baixo
`i_key[127:0]`, `i_key_load`	I	Pulso carrega a chave e dispara a expansão
`o_key_ready`	O	`1` = *round keys* válidas
`i_start`, `i_decrypt`, `i_block[127:0]`	I	Pulso inicia 1 bloco; `i_decrypt` = 0 cifra, 1 cifra inversa
`o_block[127:0]`	O	Resultado; mantido até o próximo `i_start`
`o_busy`	O	Bloco em processamento
`o_done`	O	Pulso de 1 ciclo ao final

Os sinais `i_start` e `i_key_load` são ignorados enquanto `o_busy` ou a expansão estiverem ativos; `i_start` também é ignorado se `o_key_ready = 0`.

4.2 Escalonamento (1 rodada por ciclo)

O escalonamento das operações por ciclo de processamento encontra-se detalhado na Tabela 3.

Latência: 11 ciclos entre a borda do `i_start` e o `o_done`. O `dp` calcula os dois caminhos em paralelo (16 S-boxes + 16 S-boxes inversas) e o registrador `dec_q` seleciona o resultado; o contador de rodada percorre $1 \rightarrow 10$ ou $9 \rightarrow 0$. O índice de *round key* (`rk_sel`) vale `round` durante o processamento e `rk[0]` ou `rk[10]` na carga inicial.

Tabela 3: Escalonamento ciclo a ciclo no datapath (1 rodada por ciclo).

Ciclo	Cifra	Cifra inversa (FIPS-197, Fig. 12)
0	`state` = `in` $\oplus$ `rk[0]`	`state` = `in` $\oplus$ `rk[10]`
1 . . . 9	`SubBytes`, `ShiftRows`, `MixColumns`, `AddRoundKey(rk[r])`, $r = 1 \ldots 9$	`InvShiftRows`, `InvSubBytes`, `AddRoundKey(rk[r])`, `InvMixColumns`, $r = 9 \ldots 1$
10	`SubBytes`, `ShiftRows`, `AddRoundKey(rk[10])`	`InvShiftRows`, `InvSubBytes`, `AddRoundKey(rk[0])`

5 Verificação

5.1 Estratégia

Os *testbenches* são autoverificáveis (terminam em `PASS`/`FAIL`) e usam vetores gerados por um modelo Python próprio. O modelo foi checado contra os valores da FIPS-197 (Apêndices B e C.1, inclusive `rk[10]` de C.1) e contra a biblioteca `cryptography` em 200 vetores aleatórios antes de gerar os vetores do RTL. Os resultados consolidados estão na Tabela 4.

Tabela 4: Resumo da suíte de verificação e resultados obtidos.

Testbench	O que verifica	Resultado
`tb_aes_key_exp`	11 *round keys* dos Apêndices B e C.1, leitura em ordem direta e inversa, latência = 10 ciclos, estado pós-*reset*	`PASS` (0 erros)
`tb_aes_dp`	C.1 e B, cifra e decifra; 64 chaves/blocos aleatórios nos dois sentidos; `o_block` mantido após `o_done`; `i_start` ignorado sem chave	`PASS` (0 erros)
`tb_aes_round_transforms`	(regressão das Semanas 2–3) S-box nas 256 entradas, todas as rodadas dos Apêndices B e C.1, inversas	`PASS` (0 erros)

5.2 Vetores de referência (todos conferem)

A Tabela 5 apresenta os vetores oficiais da FIPS-197 utilizados na validação. Cada vetor é verificado nos dois sentidos: $E(\text{pt}) = \text{ct}$ e $D(\text{ct}) = \text{pt}$.

5.3 Robustez do *testbench*

Foi feito um teste de mutação: alterar um coeficiente de `InvMixColumns` (`mul11` $\rightarrow$ `mul13`) fez os testes falharem nas rodadas da cifra inversa; o RTL original foi restaurado e a

Tabela 5: Vetores de referência da norma FIPS-197 verificados em cifra e decifra.

Caso	Chave	Texto claro	Cifrado
Apêndice C.1	000102030405060708090a0b0c0d0e0f	00112233445566778899aabbccddeeff	69c4e0d86a7b0430d8cdb78070b4c55a
Apêndice B	2b7e151628aed2a6abf7158809cf4f3c	3243f6a8885a308d313198a2e0370734	3925841d02dc09fbdc118597196a0b32

regressão voltou a passar.

5.4 *Lint*

A execução de `verilator -lint-only -Wall` sobre todos os módulos (topo `minisoc_aes_dp`) encerrou **sem erros e sem *warnings*** (suprimidos apenas `DECLFILENAME`, porque o pacote `minisoc_aes_gf_pkg` está em arquivo próprio, e `UNUSEDSIGNAL`).

5.5 Como reproduzir

Para executar o fluxo completo de verificação (requer Verilator $\geq$ 5.0 e Python 3):

```
./scripts/run_tests.sh          # lint + os três testbenches, retorna != 0 se algo
python3 scripts/gen_aes_ref.py  # (opcional) regenera S-boxes e vetores
```

6 Decisões de projeto e consequências para as próximas semanas

- O `aes_top` usará `i_key_load` uma vez por `START` e só então `i_start` por bloco; os modos CFB/OFB/CTR usam apenas `i_decrypt = 0`.
- O `dp` não faz nenhuma operação de encadeamento; operações de XOR com IV/*chaining*/contado ficam no `aes_top` (seção 3 da microarquitetura).
- Custo por bloco no `aes_top`: 11 ciclos do `dp` + 5 de leitura + 4 de escrita + controle $\approx$ 23 ciclos.

7 Limitações

- Um caminho de rodada em 1 ciclo (S-box + MixColumns + XOR) é otimista para `sys_clk` = 2 GHz em silício; aceitável porque o PLL é um modelo de integração (LIM-PLL-01). Trata-se de verificação funcional, não de *timing*.
- Os dois caminhos (cifra e inversa) duplicam S-boxes; um compartilhamento via S-box composta ficou como otimização futura.
- Não há tratamento de chave alterada durante o processamento (ignorada por protocolo); o `aes_top` deve garantir essa condição.

8 Conclusão

Os módulos `minisoc_aes_key_exp` e `minisoc_aes_dp` foram implementados conforme a microarquitetura v1.0. Cifra e decifra conferem com os Apêndices B e C.1 da FIPS-197 e com 64 vetores aleatórios, o *testbench* roda com um único comando e o *lint* não reporta erros — atendendo ao critério de aceite da Semana 4.